

Rotate an array -

Rotate an array by $k = 3$ (rotate 3 times or by 3 elements)

Index	0	1	2	$k = 3$	4	5	6	7	8
nums []	1	2	3	4	5	6	7	8	9
$k = 1$	9	1	2	3	4	5	6	7	8
$k = 2$	8	9	1	2	3	4	5	6	7
$k = 3$	7	8	9	1	2	3	4	5	6

Programmatically rotation can be done in 3 steps -

- step 1 - reverse the entire array, reverse $(0, n - 1)$
- step 2 - reverse $(0, k - 1)$
- step 3 - reverse $(k, n - 1)$ - where n is the length of the array.

Index	0	1	2	$k = 3$	4	5	6	7	8
nums []	1	2	3	4	5	6	7	8	9
step 1	9	8	7	6	5	4	3	2	1
step 2	7	8	9	6	5	4	3	2	1
step 3	7	8	9	1	2	3	4	5	6

Problem -

This logic works fine until k is less than length of the array. ($k < n$).

When $k > n$ ($k = 7, n = 5$) then step 2 which is reverse $(0, k - 1) \rightarrow$ reverse $(0, 6)$ will throw out of bound exception as index $[6]$ is out of range for an array of length 5.

Solution to this problem is to normalize k -

$k = k \% n$, where n is length of array.

ex. nums = [1, 2, 3, 4, 5] $\#n = 5$

- Rotate by $k = 6$ **is same as** rotate by $(k \% n) \rightarrow (6 \% 5) \rightarrow 1$, $k = 1$
- Rotate by $k = 8$ **is same as** rotate by $(k \% n) \rightarrow (8 \% 5) \rightarrow 3$, $k = 3$
- Rotate by $k = 5$ **is same as** rotate by $(k \% n) \rightarrow (5 \% 5) \rightarrow 0$, $k = 0$ (full rotation)

Normalization avoids full rotations & avoid index out of bound.